

Construir un proxy

CC4303 Redes · Sección Ivana Bachmann

Hector Bonilla V. • Lázaro Narváez U.

Repositorio: <https://github.com/LexfordMax14/redes-actividades-LN-HB>

1. Declaración de uso de IA

Modelo	Integrante	Dónde se usó
Gemini	Lázaro	Consultas de sintaxis de Python sobre <code>.items()</code> y <code>str.replace()</code> . Se usó para aprender el lenguaje y no para decisiones de diseño.
Claude (Opus 5)	Lázaro	Diagnóstico de errores en <code>recv_message</code> y migración del body de <code>str</code> a <code>bytes</code> . Ejecución de las pruebas contra el servidor del curso. Estructura de este informe.
Claude (Sonnet)	Héctor	Consulta sobre el significado de <code>b</code> en sockets durante la depuración de un error. Revisión de código.

2. Cómo ejecutar

```
python tcp_socket_server.py
```

- El programa debe ejecutarse desde la carpeta del repositorio. Los archivos `rules.json` y `403.jpg` se leen con ruta relativa.
- El proxy escucha en el puerto 8000. La dirección se define en la variable `IP_VM` de `tcp_socket_server.py`.
- El comportamiento se configura en `rules.json` mediante tres campos: usuario, dominios bloqueados y palabras prohibidas.

Para una prueba rápida:

```
curl http://cc4303.bachmann.cl/ -x IP_VM:8000
```

Entorno. El proxy fue probado en máquina virtual usando `IP_VM` y en localhost usando 127.0.0.1.

3. Diagrama del proxy

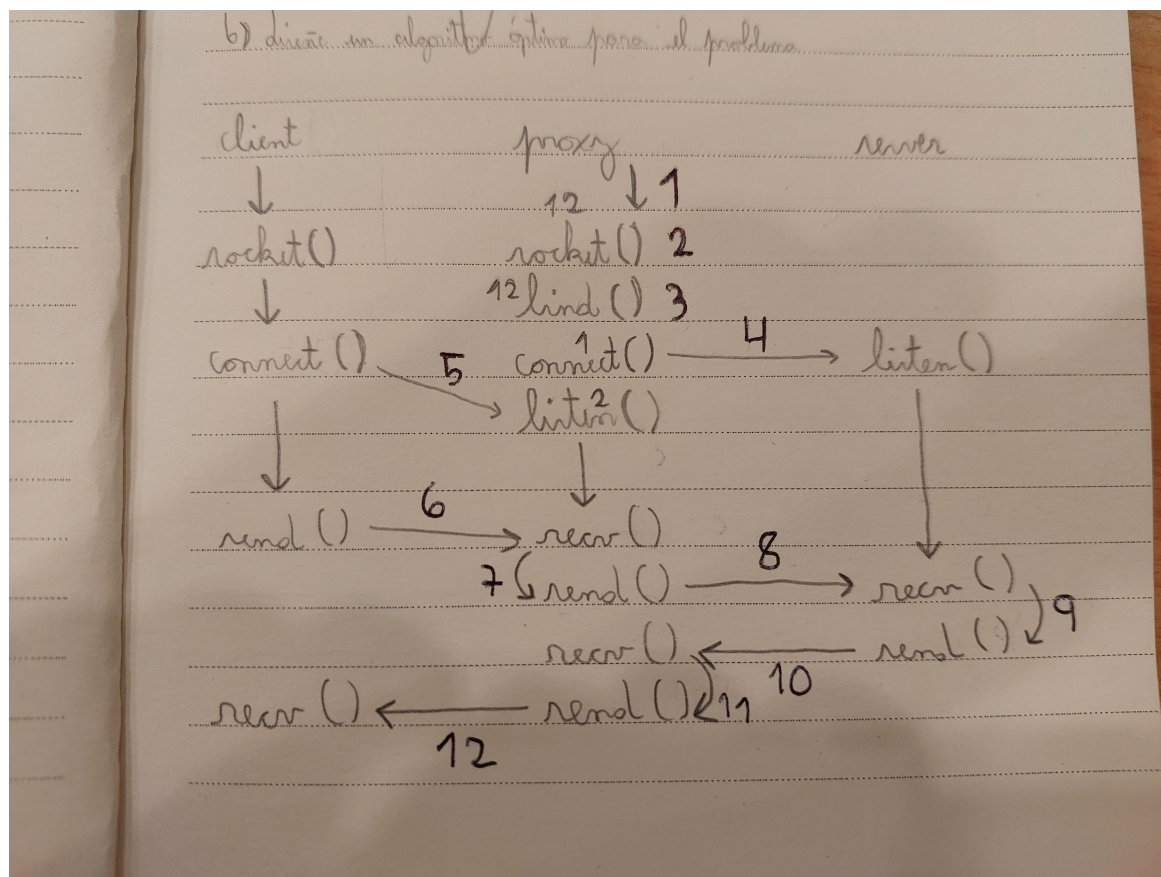


Figura 1: Flujo de llamadas entre cliente, proxy y servidor.

El proxy necesita **tres sockets** para atender una transacción.

Socket	Rol
server_socket	Recibe conexiones mediante <code>accept()</code> . No transfiere datos.
new_socket	El proxy actúa como servidor frente al cliente. Se crea uno por conexión.
proxy_socket	El proxy actúa como cliente frente al servidor real.

La idea central es que un proxy cumple ambos roles de forma simultánea. Recibe conexiones con `accept()` y las origina con `connect()`.

4. Diseño

4.1 Arquitectura

Separación en dos archivos. El código se dividió siguiendo el principio de separación de responsabilidades visto en Ingeniería de Software. El archivo `parser.py` implementa el protocolo HTTP: lee del socket, interpreta mensajes y los reconstruye. El archivo `tcp_socket_server.py` implementa la política del proxy: a quién conectarse, qué bloquear y qué censurar. Ninguno de los dos necesita conocer el interior del otro.

Clase `Http_HL` en lugar de un diccionario o una tupla. Se optó por una clase para que el editor respetara las firmas y ofreciera autocompletado. La clase además garantiza una estructura

fija donde `start_line`, `head` y `body` siempre existen y conservan su tipo. Esto elimina los chequeos defensivos en el resto del código.

4.2 Qué extrae `parse_HTTP_message`

La función divide el mensaje en tres partes usando `\r\n\r\n` como separador.

Campo	Tipo	Contenido
<code>start_line</code>	<code>str</code>	GET http://host/ruta HTTP/1.1. Contiene método, path y versión
<code>head</code>	<code>dict[str, str]</code>	Un par por cada header
<code>body</code>	<code>bytes</code>	Contenido crudo

Las decisiones detrás de esta estructura son dos.

- **Los headers se guardan como `str` y el body como `bytes`.** La especificación define los headers como ASCII, mientras que el body puede ser binario como en el caso de un JPEG. Decodificarlo asumiendo UTF-8 provoca un error con cualquier imagen.
- **Se usa `partition(b":")` y no `split`.** La primera corta únicamente en el primer `:` del header. Esto importa porque valores como `Date: Thu, 27 Aug 2026 17:07:08 GMT` contienen varios.

La función `create_HTTP_message` es la inversa exacta. Reconstruye `start line`, headers, `\r\n` y `body`, y devuelve el mensaje en `bytes`.

4.3 Bloqueo de dominios

El proxy toma la URL que el cliente escribe en la `start line` y verifica si alguna entrada de la lista `blocked` aparece dentro de ella. Si aparece responde 403 y cierra la conexión. Si no, la petición sigue por el camino normal hacia el servidor.

- **Por qué se usa la `start line` y no el header `Host`.** El `Host` es un metadato del header y contiene solo el dominio, nunca lo que viene después del slash. Con él una entrada como `cc4303.bachmann.cl/secret` no puede calzar jamás, de modo que la única forma de bloquear `/secret` era bloquear el dominio completo. Eso dejaba inaccesibles también `/` y `/replace`. La `start line` en cambio trae la URL completa cuando el cliente pasa por un proxy, y eso permite distinguir una ruta de otra. Por esa razón `/replace` sigue funcionando mientras `/secret` queda bloqueado.
- **La comparación es por substring y no por igualdad.** La decisión es deliberada. Si una página está bloqueada sus rutas descendientes también deben estarlo, como ocurre con `/secret/algo`. El efecto lateral es que se bloquean rutas que comparten prefijo sin ser descendientes, por ejemplo `/secretos-publicos`. Se aceptó porque en un control parental resulta preferible bloquear de más que de menos.
- **Respuesta ante una URL bloqueada.** El proxy devuelve 403 Forbidden junto a un HTML que incluye la etiqueta `<img src=/403.jpg>`.
- **La ruta `/403.jpg` se intercepta antes del chequeo de bloqueo.** La imagen es un recurso local y no del servidor remoto. Si se evaluara después, el proxy la buscaría afuera y obtendría un 404.
- **La imagen se sirve con 200 OK y no con 403.** El recurso prohibido es la página y no la imagen. El 403 corresponde al HTML bloqueado, mientras que la petición posterior de `/403.jpg` es legítima y el proxy la satisface localmente.

¿Cuántos ciclos HTTP se necesitan para mostrar una imagen en un navegador?

Se necesitan dos. El primero entrega el HTML y el segundo la imagen. El navegador no puede saber que existe una imagen hasta recibir y parsear el HTML, por lo que ambas peticiones son necesariamente secuenciales y no pueden unificarse.

4.4 Reemplazo de palabras

El proxy recorre `forbidden_words`, que es una lista de diccionarios de un par cada uno, y aplica `bytes.replace()` sobre el body por cada par. Los reemplazos son acumulativos porque cada iteración opera sobre el resultado de la anterior.

- **No se utiliza `re`.** El enunciado permite únicamente `socket`, `json` y `sys`. El método `replace()` resulta suficiente porque las claves del JSON no se solapan entre sí y coinciden en mayúsculas con el contenido de las páginas del curso. Un enfoque insensible a mayúsculas habría requerido expresiones regulares.
- **Se recalcula `Content-Length` tras cada reemplazo.** El body cambia de tamaño porque proxy ocupa 5 bytes y `[REDACTED]` ocupa 10. Como el body se almacena en `bytes`, el nuevo valor es simplemente `len(body)`. El largo debe medirse en bytes y no en caracteres, ya que las páginas contienen tildes.

4.5 Modificación de headers

El proxy agrega el header `X-ElQuePregunta` a la request que envía al servidor. Su valor se toma de `rules.json`, lo que deja el nombre parametrizado fuera del código.

4.6 Mensajes más grandes que el buffer

La función `recv_message` lee en dos fases y cada una responde a una de las preguntas planteadas.

¿Cómo sé que el HEAD llegó completo?

El criterio es el delimitador `\r\n\r\n`. La función acumula lo recibido hasta encontrarlo.

¿Qué pasa si los headers no caben en mi buffer?

No representa un problema. La lectura no se limita a un buffer sino que acumula dentro de un `while` hasta detectar el delimitador. El mecanismo funciona incluso con `buff_size=1`.

¿Y el BODY?

El criterio es `Content-Length`. La función lee hasta que `len(body)` alcanza ese valor.

¿Cómo sé si llegó el mensaje completo?

Combinando ambos criterios: el delimitador para el head y `Content-Length` para el body.

Existe un detalle que condiciona toda la implementación. Un mismo `recv` puede traer el final del head junto con el inicio del body. Ese sobrante debe conservarse y contarse dentro de `len(body)`. Descartarlo lleva al proxy a pedir bytes de más y quedarse bloqueado.

Por qué el buffer por defecto es 4 y no 50. El valor busca forzar el peor caso en cada ejecución. Con buffers grandes los errores de lectura no se manifiestan. Durante el desarrollo apareció exactamente el caso descrito arriba, donde un `recv` traía el head más un fragmento del body y ese fragmento se perdía. Con un buffer de 50 ese efecto podría no haberse observado nunca.

5. Pruebas

5.1 Con curl

Prueba	Comando	Resultado
Bloqueo	<code>curl .../secret -x IP_VM:8000</code>	403
Imagen	<code>curl .../403.jpg -x IP_VM:8000</code>	200, image/jpeg, 20134 bytes
Header	<code>curl .../ -x IP_VM:8000</code>	Bienvenide HL!, valor tomado de rules.json
Reemplazo	<code>curl .../replace -x IP_VM:8000</code>	9 [REDACTED], 5 [FORBIDDEN] y 3 [???]. No queda ninguna palabra sin censurar
Content-Length	<code>curl -i .../replace -x IP_VM:8000</code>	El header declara 1225 bytes y el body mide 1225

5.2 Tamaño de buffer

Para el servidor del curso el head mide 192 bytes, la start line 17 y el mensaje completo 1351.

buff_size	Caso	Resultado
200	Menor que el mensaje y mayor que los headers	Correcto
50	Menor que los headers y mayor que la start line	Correcto
4	Menor que la start line	Correcto
1	Mínimo posible	Correcto

En los cuatro casos el body llega completo y su largo coincide con **Content-Length**. Los dos primeros corresponden a los escenarios exigidos por el enunciado.

5.3 Con el navegador

Al acceder a `http://cc4303.bachmann.cl/secret` el contenido del sitio deja de verse. El proxy responde 403 y el navegador muestra la imagen alojada localmente. El log registra los dos ciclos HTTP descritos antes: uno para el HTML y otro para la imagen.

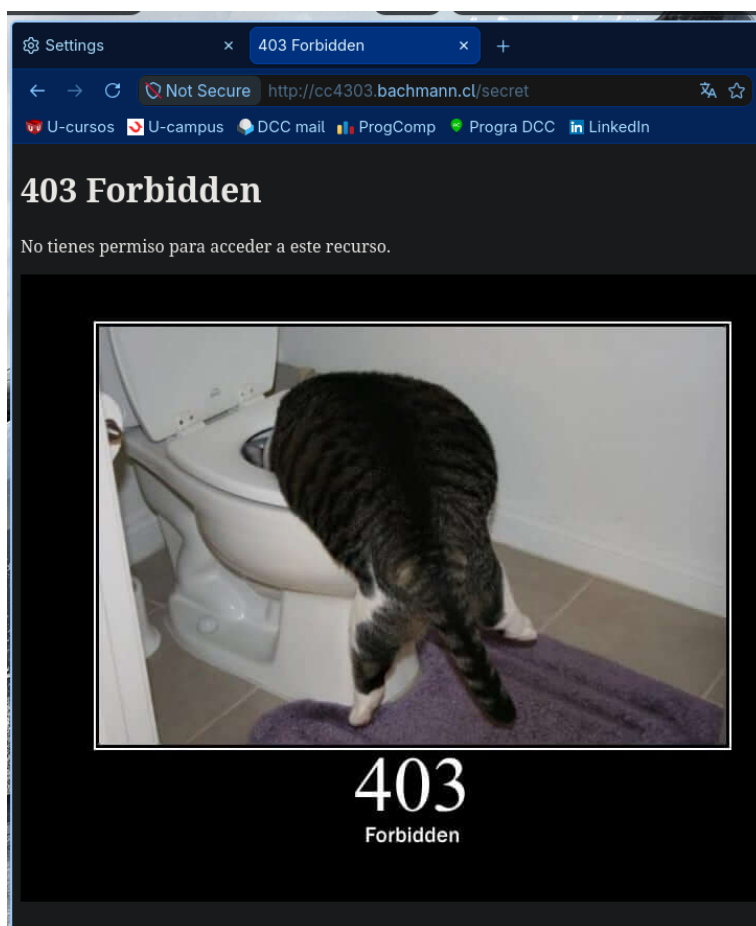


Figura 2: /secret bloqueado. El proxy responde 403 y sirve la imagen local.

En `http://cc4303.bachmann.cl/` el título muestra **Bienvenide HL!**. Ese valor lo toma el servidor del header `X-ElQuePregunta` que agrega el proxy. El enlace de la página aparece censurado.

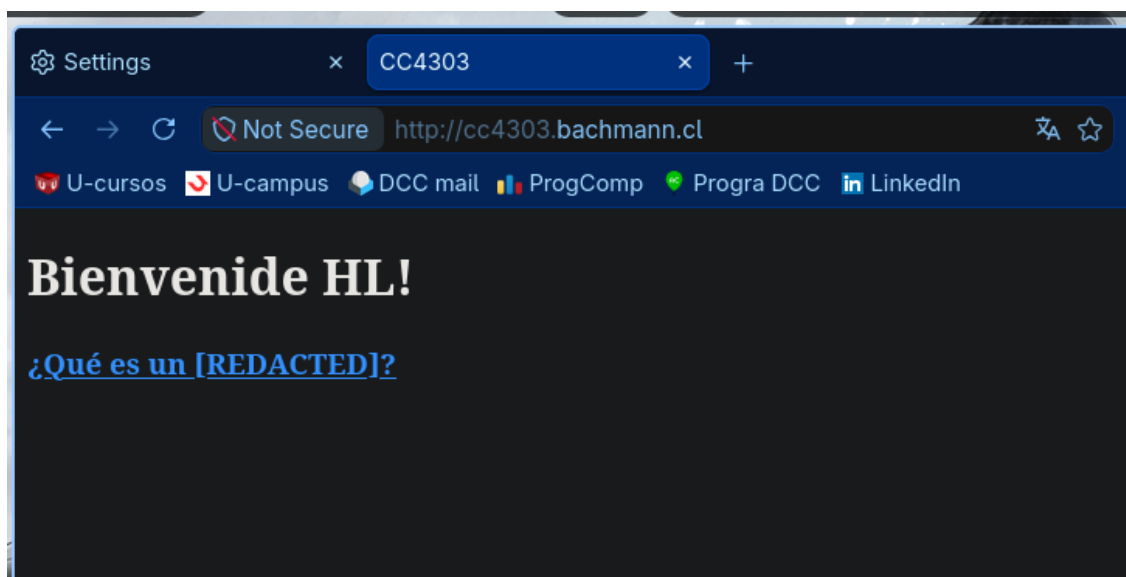


Figura 3: Efecto del header `X-ElQuePregunta` sobre el mensaje de bienvenida.

En `http://cc4303.bachmann.cl/replace` el texto es el mismo que sin proxy salvo por las palabras reemplazadas. No hay cortes ni caracteres rotos y las tildes se conservan, lo que confirma que `Content-Length` se recalcula en bytes. El `<title>` de la pestaña también queda censurado.

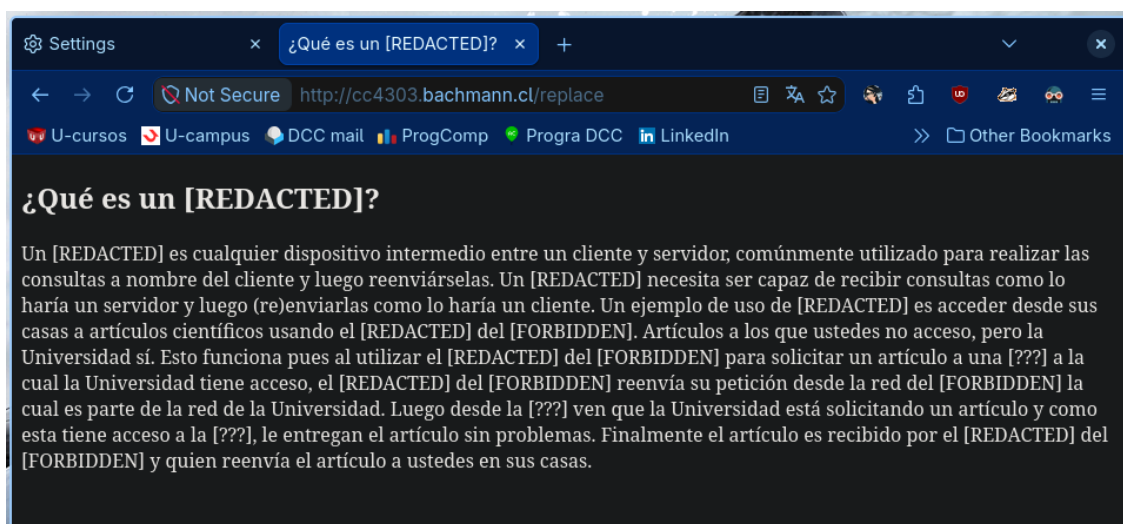


Figura 4: Palabras prohibidas censuradas en /replace, incluido el título de la pestaña.